



Smart HAS

ComunidadeConectada

Fase 5 · Capítulo 1 — Mobile Hybrid App e a Sociedade 5.0

Integrante: **Yuri Sousa**
RM: **556163**

O que é o Smart HAS

Um app híbrido de economia colaborativa para a Sociedade 5.0



Comunidade conectada

Pessoas oferecem e pedem recursos — ferramentas, saúde, livros, alimentos — por empréstimo, troca ou doação.



Geolocalização

Recursos aparecem no mapa por proximidade, facilitando encontros presenciais na vizinhança.



ConectaIA

Assistente de IA (Gemini) ajuda a melhorar descrições de ofertas e tira dúvidas dos usuários em tempo real.

O ponto de partida (Fase 4)

Um protótipo Flutter completo na interface — mas sem nenhum dado real por trás

O que já existia

- 11 telas funcionais: onboarding, login, home, mapa, chat, chatbot, oferta, perfil
- Arquitetura com Provider/ChangeNotifier
- Integração com IA (Gemini) para chat
- UI consistente e responsiva

O que faltava

- Usuários, recursos e mensagens 100% mockados em memória
- Nada persistia — reiniciar o app apagava tudo
- Sem autenticação real, sem backend, sem banco de dados
- Botão "Publicar Oferta" nem sequer salvava o recurso

Parte 1 — Escolha da Stack Mobile



React Native (escolhido)

- 5 dos 13 capítulos da fase são só sobre React Native
- Opção A do enunciado pede View/Text/Image/Button + React Navigation, ao pé da letra
- Node.js já disponível; Expo permite montar e testar rápido
- Hooks (useState/useEffect) e AsyncStorage seguindo os padrões dos Cap. 05 e 06



Flutter (versão anterior)

- 11 telas da Fase 4, já integradas ao backend real
- Mantida no repositório como histórico do projeto
- Não usa nenhum conteúdo específico ensinado nesta fase
- Base de referência para comparar a migração

Roadmap Tecnológico



Fases 1–4

Escopo, arquitetura e protótipo Flutter
com dados mockados



Fase 5 · Cap. 1 (agora)

Backend Spring Boot real, integração
mobile + dashboard Angular



Próximas fases

Upload de imagens, sessão persistente,
IA de logística, deploy em nuvem

Parte 2 — Backend Spring Boot

Java 21 · Spring Boot 4 · Firebase (Cap. 13) · API REST completa para o app e o dashboard



Firebase Authentication

Cadastro/login reais; senha nunca fica no backend



Firestore

Banco NoSQL na nuvem, CRUD completo



3 entidades

User, Resource e Message, com DTOs dedicados



Controller → Service → Repository

Camadas bem definidas, injeção por construtor



Tratamento de erros padrão

400 / 401 / 403 / 404 / 409 em formato único



Swagger / OpenAPI

Documentação interativa em /swagger-ui

API REST — Cobertura e Verificação

18

endpoints REST

3

entidades persistidas

100%

testado via curl ponta a ponta

Principais rotas

- POST /api/auth/register, /api/auth/login
- GET/PUT /api/users/me · GET/DELETE /api/users (ADMIN)
- GET/POST/PUT/DELETE /api/resources
- GET/POST /api/messages, /api/messages/conversations
- GET /api/admin/stats (ADMIN)

Verificado de verdade

- Servidor compilado e rodando localmente (./mvnw spring-boot:run)
- Cadastro real no Firebase Auth + login com as mesmas credenciais, em chamadas separadas
- Regras de permissão dono-vs-admin conferidas na prática
- Bug real no SDK do Firebase ("Not in GZIP format") encontrado e contornado

Parte 3 — Dashboard Angular

Painel administrativo consumindo a mesma API do app mobile



Rotas

/home (pública), /login e /admin (protegida por AuthGuard)



HttpClient + Interceptor

Anexa automaticamente o token JWT em toda chamada à API



4 formas de binding

{{ }} interpolação · [] property · () evento · [()] ngModel



***ngIf / *ngFor**

Estados de carregamento/erro/vazio e listas de usuários e recursos

Integração Ponta a Ponta

As três aplicações compartilham o mesmo contrato de API e os mesmos dados reais



Evidência: um recurso renomeado via API apareceu, ao vivo, atualizado no dashboard Angular.

Conclusão

O Smart HAS saiu de um protótipo Flutter com dados mockados para um sistema full-stack real e alinhado ao conteúdo da fase: app React Native, backend Spring Boot com Firebase (Authentication + Firestore), e dashboard Angular — todos integrados pelo mesmo contrato de API.

Entregáveis desta atividade

- Código-fonte: GitHub — [link a inserir]
- Vídeo de demonstração (YouTube não listado) — [link a inserir]
- Relatório completo em PDF (documento em anexo)

Obrigado!

Yuri Sousa · RM 556163